Qawafel Graduate Software Engineer Assessment

Wholesaler Financing Insights
Written requirements, technical design, and integration judgment

Prepared by: Mishal Saad

Scope and key decisions

The working prototype turns a fictional wholesaler's sales invoices into a preliminary financing evidence package. It validates uploads, computes traceable financial indicators, highlights unusual invoices using unsupervised learning, and exports a reviewer summary. It does not decide creditworthiness.

The production design below covers the full scenario, including bank-statement processing and safe bank submission. Those production services are proposed, not implemented by the Streamlit prototype. All sample business and customer records are fictional.

Task 1 - Requirements

Users and acceptance criteria

1. Applicant: submit a complete request. As an authorized wholesaler representative, I want to supply 12 months of sales invoices and bank statements so that Qawafel can prepare my financing request.

- Only an authenticated representative of the owning business can create or view its request.
- The interface states the requested period, supported formats, and permitted uses of data before upload; consent is recorded.
- Valid Excel invoices and PDF statements receive document references and processing states. Unsupported or oversized files return actionable errors.
- Missing months are shown explicitly. The applicant can explain genuine zero activity rather than fabricate transactions.

2. Applicant: correct data without losing the request. As an applicant, I want specific validation feedback so that I can fix a file rather than restart the application.

- Invalid dates, missing fields, duplicate invoice keys, unsupported currencies, and inconsistent amounts identify their source row or page.
- A failed document cannot contribute to a submission-ready package. Corrections create a new version and invalidate any prior approval of the changed package.
- The application preserves an audit trail of which version was reviewed and submitted.

3. Qawafel reviewer: verify financial evidence. As a financing reviewer, I want sales, receivables, and bank receipts kept distinct so that I can assess evidence quality.

- Gross invoice value equals recorded settlements plus outstanding value for the prototype's full-payment cohort.
- Overdue value is a subset of outstanding, calculated against a visible snapshot date.
- In production, extracted bank transactions retain page references. Reconciliation permits partial and many-to-one matches and requires confirmation for ambiguous matches.
- Unmatched receipts are not silently labeled sales; internal transfers, refunds, and loans require classification.

4. Qawafel reviewer: prepare a review package. As a reviewer, I want a versioned summary and source references so that the bank can verify the request efficiently.

- The package includes period coverage, key indicators, data-quality exceptions, unresolved questions, assumptions, and source references.
- AI flags remain review suggestions, separate from deterministic metrics and bank decisions.
- A reviewer must resolve or explicitly accept documented exceptions before approving a specific package version for submission.

5. Qawafel operator: submit once and reconcile safely. As an operator, I want retries and uncertain responses handled safely so that the business does not receive duplicate applications or false final statuses.

- Repeating a submission command with the same idempotency key and identical body returns the original submission; a changed body returns a conflict.
- A timeout after sending becomes an unknown outcome, not a failure that automatically triggers resubmission.
- A contradictory bank status is retained as evidence and routed for verification before a final status is exposed.

6. Bank reviewer: receive an understandable package. As a bank reviewer, I want consistent, traceable financial evidence so that I can assess financing without guessing what the figures mean.

- Each accepted submission has a stable internal reference, package version, reporting period, and currency.
- Net invoiced sales, gross amounts, recorded settlement, and bank-confirmed receipts have distinct definitions and provenance.
- Exceptions and the evidence still needed are visible. The bank retains its lending decision.

Questions before building

1. Who submits, owns, and approves the request? Can an accountant act for the business, and what consent is required?
2. Is financing for inventory purchases, receivables, or general working capital? Which documents and ratios does the bank actually require?
3. Are the invoices sales or purchases? Do they include VAT, credit notes, returns, partial settlements, multiple currencies, or multiple business entities?
4. What does a blank payment date mean, and how reliable are recorded settlement dates? Are there opening receivables from outside the review period?
5. What Excel templates and bank PDF layouts occur? Are statements scanned, password-protected, or from multiple accounts?
6. What counts as 12 months: calendar months or a rolling period? How should zero-trading months and incomplete statements be represented?
7. How are supplier payments, owner injections, transfers, refunds, and customer receipts distinguished during reconciliation?
8. What bank status is authoritative, what exactly does the reported 3% error mean, and which lookup/reconciliation channels exist?
9. What are expected volumes, turnaround targets, retention rules, correction procedures, and escalation ownership?

Data risks and responses

Risk	Practical response
Duplicated/re-exported invoices inflate sales	Unique business/invoice keys plus document hashes; report duplicates and resolve before summing. Do not silently deduplicate conflicting records.
Mixed date formats, currencies, or units	Explicit schema, ISO dates, SAR-only prototype, two-decimal validation; reject ambiguous conversions.
Gross value confused with revenue or cash	Separate net, VAT, gross, recorded payments, and bank-confirmed receipts. Publish metric definitions.
Partial payments, returns, and credit notes	Reject unsupported prototype fields; production payment allocations and linked credits must preserve balances.
Missing months, biased invoice selection, or forged files	Coverage and provenance checks, applicant confirmation, bank reconciliation, and human review; file parsing alone cannot prove authenticity.
OCR mistakes or statement balances that do not reconcile	Retain page lineage, confidence and balance checks; route exceptions to a reviewer.
Transfers counted as customer income	Classification and matching with evidence; never equate every bank credit with sales.
Sensitive files or malicious uploads	Authorization, file limits and content checks, isolated parsing and malware scanning in production, encryption and restricted access; fictional data only in this demo.

Task 2 - Technical specification

Architecture and data model

Use a small production web/API service, a background processing worker, PostgreSQL, encrypted object storage, and an isolated bank adapter. The worker handles document parsing and retries independently of the upload request. This is a future design; the prototype intentionally uses a single Python/Streamlit process with no database.

Primary keys are UUIDs. Money is stored as integer minor units (`BIGINT`, halalas for SAR), with an explicit currency. Dates are `DATE`; event times are UTC timestamps. Every application-scoped query is authorized against its business owner. Financial snapshots and approved package versions are immutable.

Entity	Main fields and types	Relationships / constraints
Business	id UUID PK; legal_name TEXT; registration_reference TEXT; owner_subject TEXT	Owns applications. No real business identifiers in demo data.
FinancingApplication	id UUID PK; business_id UUID FK; period_start/end DATE; snapshot_date DATE; currency CHAR(3); workflow_state TEXT; consent_at TIMESTAMPTZ; version INT	End >= start; consent before bank submission. Version guards concurrent edits.
Document	id UUID PK; application_id UUID FK; kind TEXT; object_key TEXT; sha256 CHAR(64); version INT; parsing_state TEXT; parser_version TEXT; supersedes_id UUID nullable FK	Kind is invoices or statement. Immutable source object; only active approved versions enter a package. Hash uniqueness is scoped to application and purpose.
Invoice	id UUID PK; application_id UUID FK; document_id UUID FK; external_id TEXT; customer_external_id/name TEXT; issue/due DATE; net/vat/gross_minor BIGINT; currency CHAR(3); source_row INT	Unique active application/external_id. Net + VAT = gross; amounts and date order validated. These are applicant sales.
BankTransaction	id UUID PK; document_id UUID FK; account_token TEXT; transaction_reference TEXT nullable; booked_on DATE; signed_amount_minor BIGINT; currency CHAR(3); source_page INT	Positive/negative sign is normalized from the bank format. Reference may be absent; hash alone is not sufficient proof of duplication.
PaymentAllocation	id UUID PK; invoice_id UUID FK; transaction_id UUID FK; allocated_minor BIGINT; confirmed_by TEXT; confirmed_at TIMESTAMPTZ	Supports partial and many-to-many matching. Allocations cannot exceed invoice balance or available transaction value. Confirmation is audited.
ReviewPackage	id UUID PK; application_id UUID FK; version INT; content_hash CHAR(64); artifact_key TEXT; approved_by TEXT nullable; approved_at TIMESTAMPTZ nullable	Includes source/version references, calculations and exceptions. Submission requires approval of this exact hash.
BankSubmission	id UUID PK; application_id UUID FK; package_id UUID FK; idempotency_key TEXT; request_hash CHAR(64); bank_reference TEXT nullable; transport_state TEXT; confirmed_status TEXT nullable	Unique application/idempotency_key. Transport outcome is separate from bank decision. Cross-application package links are forbidden.
StatusObservation	id UUID PK; submission_id UUID FK; raw_status TEXT; source TEXT; observed_at TIMESTAMPTZ; evidence_reference TEXT nullable; verification_state TEXT	Append-only evidence; a raw observation does not automatically replace confirmed status.
AuditEvent	id UUID PK; application_id UUID FK; actor_subject TEXT; action TEXT; resource_reference TEXT; created_at TIMESTAMPTZ	Append-only access/correction/approval/submission trail; exclude raw bank payloads and secrets from operational logs.

Foreign keys must reference objects within the same application/business. Corrected uploads produce a new normalized snapshot; superseded invoices are excluded rather than combined with replacements. Production credit notes would be explicit linked adjustments and require bank-agreed treatment; they are not silently represented as negative sales invoices.

Proposed API

These endpoints are a design contract, not implemented Streamlit endpoints. Assume TLS and OAuth/OIDC bearer tokens, business-scoped access, reviewer/operator roles, and least-privilege service credentials for the bank adapter. The browser never receives bank credentials.

Method and path	Purpose / response
POST /v1/applications	Create an owned request with period, currency, and consent reference. 201 with id, version, and `draft` state.
POST /v1/applications/{id}/documents	Multipart upload with kind and file. Stream to quarantined storage, verify content and limits. 202 with document id and `queued` parsing state.
GET /v1/applications/{id}/documents/{documentId}	Get processing state, missing coverage, and row/page-level validation issues. 200.
GET /v1/applications/{id}/analysis	Get versioned metrics, data quality, source references, reconciliation state, and limitations. 200; 409 if active files are not ready.
POST /v1/applications/{id}/payment-allocations	Reviewer confirms matching of receipts and invoices, with amounts and evidence. 201; reject over-allocation.
POST /v1/applications/{id}/packages	Freeze a consistent source snapshot and generate a review artifact. 202; package can be fetched using GET /v1/applications/{id}/packages/{packageId}.
POST /v1/applications/{id}/packages/{packageId}/approval	Reviewer approves the exact package hash using `If-Match` application version. 201; 409 for stale data or unresolved blocking errors.
POST /v1/applications/{id}/submissions	Operator queues one approved package; required `Idempotency-Key`. 202 with stable submission id and `queued` transport state.
GET /v1/applications/{id}/submissions/{submissionId}	Return transport state, last observation, confirmed bank status, and any verification task. Never turn `unknown` into `rejected`.

Example create request:

```
{"period_start":"2025-10-01","period_end":"2026-09-30","snapshot_date":"2026-09-30","currency"
:"SAR","consent_reference":"consent-demo-001"}
```

Example submission response:

```
{"submission_id":"sub-demo-001","transport_state":"unknown","bank_reference":null,"confirmed_s
tatus":null,"review_required":true,"message":"Delivery outcome requires reconciliation; do not
resubmit."}
```

Example validation response, HTTP 422:

```
{"error":{"code":"INVALID_INVOICE_DATA","message":"Correct the indicated
rows.","details":[{"row":14,"field":"invoice_amount_sar","reason":"Gross must equal net plus
VAT."}],"trace_id":"trace-demo-001"}}
```

Return 400 for malformed requests, 401 for missing authentication, 403 for prohibited role actions, 404 for inaccessible/nonexistent owned resources, 409 for stale versions/idempotency-body conflicts, 413 for file limits, 415 for unsupported media, and 422 for semantic errors. Use 429/503 with retry guidance only where replay is safe. An API retry by the caller must never cause a second bank submission.

End-to-end process

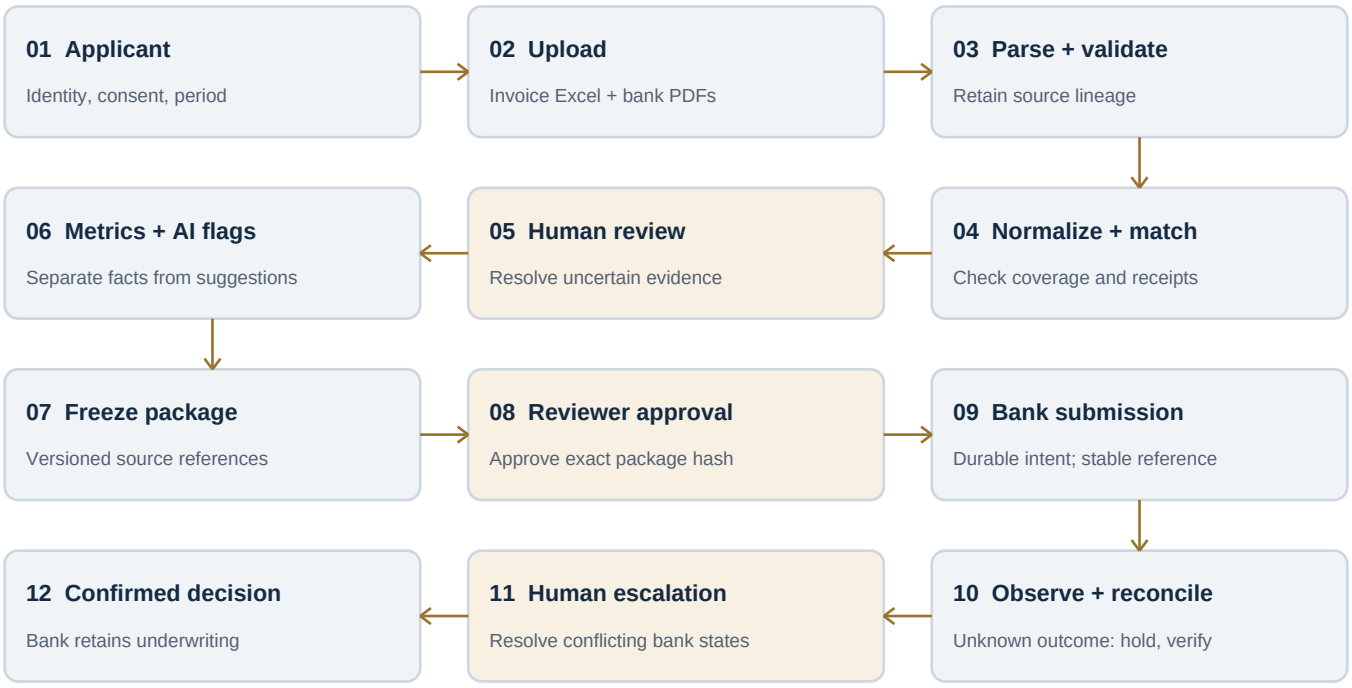

Figure 1. Proposed production flow. Validation failures return for correction. Unknown or contradictory bank outcomes remain on hold for verification; they never trigger an automatic resend. Gold-tinted steps require human judgment.

1. Authenticate business, capture consent and reporting period, upload invoice Excel and statement PDFs.
2. Quarantine and validate documents; parse in a worker with source lineage. OCR is conditional on scanned input and confidence checks.
3. Normalize invoices and transactions; check amounts, dates, coverage, duplicates, and statement balances.
4. Return blocking issues for correction. Match receipts with invoices; send ambiguous matches to human review.
5. Calculate transparent metrics and AI review flags separately. Produce a versioned package with exceptions and evidence references.
6. Reviewer approves that package; bank adapter records intent and submits safely under an agreed contract.
7. Record raw responses, reconcile uncertain/contradictory outcomes, and expose only supported confirmed decisions.

The repository's `docs/architecture.md` includes entity and process diagrams. The PDF version includes a process figure so reviewers need not render Mermaid.

Task 3 - Working AI prototype

The repository contains 667 reproducible fictional sales invoices for 24 customers across October 2025-September 2026. The app supports CSV/XLSX input, atomic validation, net/gross sales, recorded collections, receivables aging, concentration, trends, invoice-level inspection, and exportable evidence.

Deterministic financial calculations use integer halalas. The AI component is an Isolation Forest fitted independently to the current cohort: gross amount, amount relative to the customer's median, terms, and observed lateness. It produces a review queue, not a lending recommendation. The illustrative 5% review setting is not a measured fraud rate, and scores are not probabilities. Small or indistinguishable datasets skip the model. There is no labeled default or fraud dataset and no claimed predictive accuracy.

Data assumptions, reproduction steps, tests, actual verification results, and limitations are documented in the README and linked methodology files. No bank records, real bank integration, or production security are claimed by this prototype.

Task 4 - Integration judgment

Establish the contract before sending requests

First clarify whether the 3% refers to stale, contradictory, wrong-application, or incorrectly final statuses, and whether errors cluster by time or application. A successful HTTP response proves transport success, not business correctness. Repeating the same endpoint is not independent verification of a systematic error.

Agree on status meanings and permitted transitions, a correlation/reference ID, idempotency semantics, timeouts, rate limits, and the source of authoritative decisions. Ask whether lookup by our client reference, signed callbacks, decision documents, a reconciliation report, or an authorized operations contact exists. These are dependencies to confirm, not assumed bank capabilities.

Submit durably and prevent duplicates

Persist the approved package hash, our client reference, and an idempotency key before the bank call. Use a transactional outbox and one active worker per submission so process crashes cannot create a second independent attempt. Store request/response metadata and timestamps with restricted evidence references; redact credentials and financial payloads from routine logs.

Retry a submission only when it is known not to have left our system, or when the bank has agreed and implemented reliable replay protection for that exact key/body. A timeout or dropped connection after sending becomes **unknown outcome**. Query/reconcile using the existing reference; do not create a new key and resend. Our internal idempotency prevents duplicate local commands but cannot by itself guarantee the bank did not process an earlier delivery.

If the bank cannot support safe replay or lookup, unknown cases require bank operations confirmation and human resolution. This limits automatic throughput; it is safer than pretending exactly-once delivery is available.

Treat status as evidence, then reconcile

Keep raw observations separate from the last confirmed bank state. Validate references, schema, timestamps, and allowed transitions. Quarantine impossible changes, wrong-reference responses, and conflicts. Show the applicant "verification pending" rather than a falsely final acceptance or rejection.

Retry safe status reads using bounded exponential backoff with jitter within agreed rate limits. A later response can resolve ordinary staleness, but multiple matching responses alone do not prove correctness. Confirm final outcomes using the bank-agreed authoritative channel; if no independent channel exists, use manual confirmation and document the limitation. Signed delivery authenticates the sender but does not establish that the bank's decision data is correct.

Periodically reconcile pending and final submissions against the agreed authoritative evidence, not just failed HTTP calls. If a previously published final decision is contradicted, freeze automated actions, record a correction workflow, and involve bank operations. Do not silently overwrite history or trigger money movement from an unverified status.

Test and roll out without a sandbox

Build a local mock from the bank-approved schema. Exercise timeouts before/after acceptance, accepted-but-disconnected responses, duplicate commands, process crashes, malformed bodies, wrong references, delayed and out-of-order events, status reversals, rate limits, and batches of systematically incorrect statuses. A simple independent 3% random-error test is only one case and does not establish production safety.

Review contract fixtures with the bank. Request a bank-approved nonbinding test route or designated test account. If neither exists, do not send invented financing applications into ordinary production origination. Validate authorized read-only lookups on agreed reference cases first. An actual production pilot requires explicit business/bank approval, consent, tiny agreed limits, and human review of every outcome. No real customer data is used in this assessment.

Release behind a feature flag, separate read/write access, and stop new submissions when unexplained contradictions or unresolved unknown outcomes exceed the agreed operational tolerance. A circuit breaker should pause writes during an outage while preserving evidence and supporting reconciliation. Rollback stops new activity; it does not erase requests the bank may already have received.

Monitor unknown-outcome count and oldest age, status discrepancies by endpoint/time, unresolved reconciliation cases, duplicate attempts, latency, rate-limit errors, and review turnaround. Assign an on-call owner, escalation route, and reconciliation SLA with the bank. Confirm that the observed 3% is measured against a trusted outcome source; HTTP error rate is a different metric.

What I would say on the bank call

"We want to integrate safely, and we understand that around 3% of status responses may be incorrect. Could you show us the failure modes and the authoritative way to confirm a decision? In particular, can an incorrect response say approved or rejected, and can it refer to the wrong application?

"What stable client and bank references can we use? If a submission times out, can we look it up without resending? Do you guarantee idempotent replay, for how long, and what happens if the same key has a different body?

"Since there is no sandbox, can we agree on contract fixtures and a nonbinding validation route? We will not send test financing requests into normal production without an agreed procedure. Who can verify reference cases and resolve unknown outcomes?

"We propose a small, supervised rollout, retaining raw status observations and publishing final decisions only after agreed confirmation. Let's document state meanings, reconciliation evidence, rate limits, escalation owners, and stop conditions before enabling automated submission."

Task 5 - Automate the request (bonus)

First automate repeatable collection and validation. With the applicant's authorization, reuse existing platform invoices and payment records for the requested period. If files are still necessary, provide a saved mapping, completeness checklist, and incremental import rather than re-entering 12 months manually. This removes repetitive work while retaining visible data-quality gates.

Then add consented connectors. Potential sources are the customer's accounting/ERP system, invoicing platform, and bank-authorized transaction feeds where supported. Use scoped access, cursor/checkpoint-based synchronization, stable external IDs, revision tracking, and deduplication. Refresh the snapshot before submission, show what changed, and invalidate stale reviewer approval. Availability, cost, consent, and the right to reuse each source must be confirmed.

Keep humans at exceptions and commitment points. Review ambiguous receipt matches, missing periods, credits/returns, low-confidence extraction, unusual invoices, disputed balances, consent problems, and contradictory bank states. Require approval of the final evidence package. The bank continues underwriting; a connector does not make self-reported data bank verified.

Measure improvement against a baseline. Compare applicant active minutes, repeat uploads, time to a complete package, first-pass completeness, confirmed validation/reconciliation errors, reviewer minutes per case, and bank requests for clarification. Track consent withdrawals, duplicate submissions, and incorrectly published final states as guardrails. Use comparable cohorts and include exceptions; a faster average does not justify lower evidence quality. Approval rate alone is not a good success measure because applicant mix and bank policy can change.

AI tools and attribution

OpenAI Codex assisted with interpreting the brief, researching official company pages, drafting code and documentation, generating synthetic data, and running checks. ChatGPT supplied earlier background/planning context; its suggestions were checked against the original brief. The exact earlier ChatGPT model was not established. The application itself uses scikit-learn Isolation Forest locally, with no LLM API, API keys, or external financial data calls. See `docs/ai_usage.md` for scope and limitations.

Public context was checked against Qawafel's commerce and financing pages on 24 October 2026. The assessment remains the specification; bank API capabilities are unconfirmed.